Java Architecture

Created	@June 13, 2025 6:22 PM
Class	Core Java

Java Programming Language:

Java is a high-level, Object-oriented, class based programming language developed in Sun Microsystems.

Used for:

- Web Applications.
- Enterprise Systems.
- Mobile Apps.
- Backend Systems.
- API's.

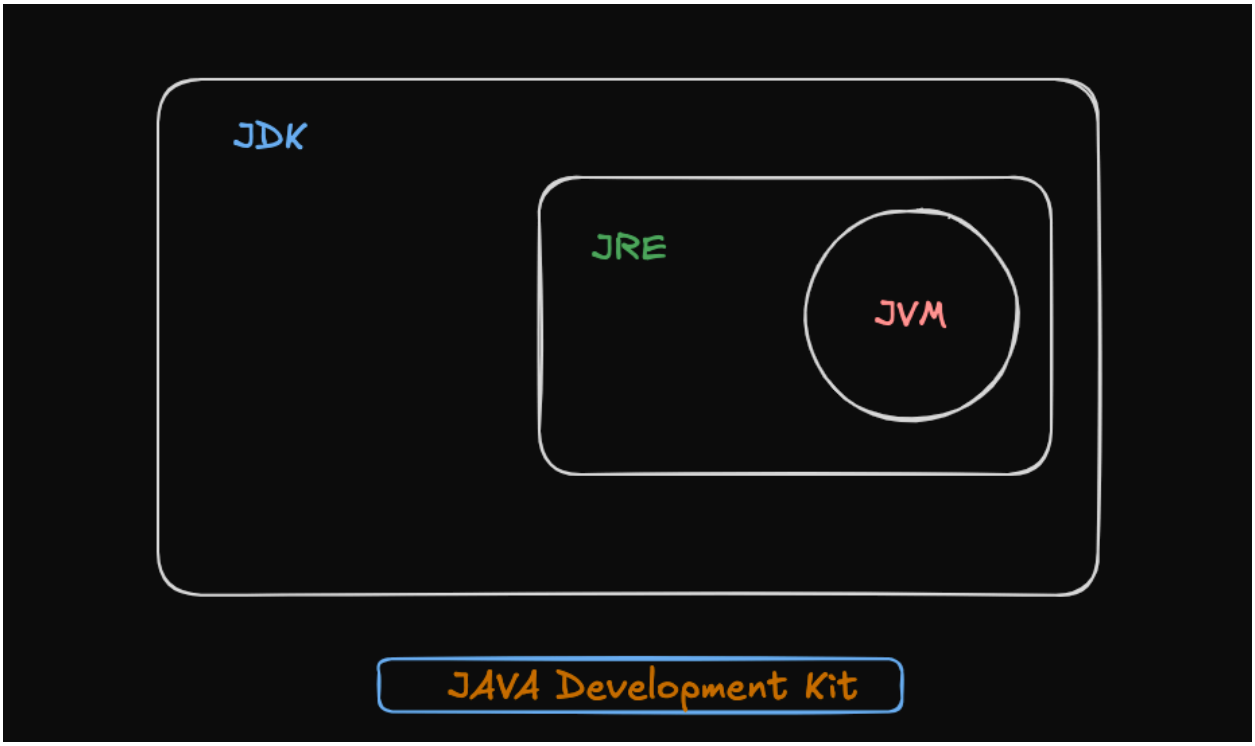
Core Features of Java:

Feature	Description
Platform Independent	"Write once, run anywhere" - Java code runs on any system with a JRE and Java Virtual Machine (JVM).
Object-Oriented	Java follows the OOP paradigm (classes, objects, inheritance, polymorphism, etc.)
Robust and Secure	Strong memory management and built-in security features.

Feature	Description
Multithreaded	Built-in support for multithreaded programming (efficient use of CPU).
Rich API	Comes with a huge set of libraries and tools.
Automatic Garbage Collection	No need for manual memory deallocation. JVM handles it.

Java Architecture :

- Java has very well though-out architecture where there are multiple components that work together to **execute java programs**, **manage and assign memory** and **maintain dependencies** along with making them platform-independent.



JDK (Java Development Kit):

When we download Java on our systems, we often download a JDK. There are many organisations that offer the JDK with little customization of their own.

- Most commonly used is OpenJDK which is free by Oracle.corp.

> JDK is a complete software development kit that comes with everything a java program needs to run.

🧩 Components of JDK:

Components	Functions
JRE (Java Runtime Environment)	Contains all necessary components and JVM that can convert a `.java` file into machine code.
Javac - compiler	compiles `.java` program files (each) into byte code(`.class`), files that can be interpreted by JVM.
Debugger, Javadocs, tools	Utilities to Test, Docs and manage code.

JRE (Java Runtime Environment):

JRE provides a runtime environment for the java code to run. It received already compiled `.java` files inform of `.class` files (byte code) that is understandable by the JVM.

- Any computer that has a JRE installed can run a java program. This is what makes Java cross-platform as the byte code can be run on any machine (Win, Mac, linux) as long as it has a JRE installed.

🧩 Components of JRE:

Component	Description
JVM (Java Virtual Machine)	Core engine that runs Java bytecode.
Class Libraries	Pre-built Java classes (like `java.util`, `java.io`).
Class Loader	Loads `.class` files into memory when needed.

JVM (Java Virtual Machine):

JVM is the Heart of Java programming language.

- JVM **interprets/compiles** the byte code into machine language, linking it with all dependencies, assign and manage memory of the program and ultimately run the source code on the machine.
- There are several components that reside within the JVM that make this happen.

🧩 Components of JVM:

- Class loader.
- Runtime Memory Handlers.
- Execution Engines.
- Garbage Collectors.

Structure:

```
JDK
├── JRE
│   ├── JVM
│   │   ├── Class Loader
│   │   ├── Execution Engine (Interpreter + JIT)
│   │   ├── Memory Areas
│   │   └── GC
│   └── Class Libraries
├── Compiler (javac)
└── Other Tools (debugger, javadoc, etc.)
```

Program Flow in Architecture:

We'll explore the **granular steps** involved in transforming human-readable source code into platform-specific machine code, detailing the key components of the JDK and how they interact.

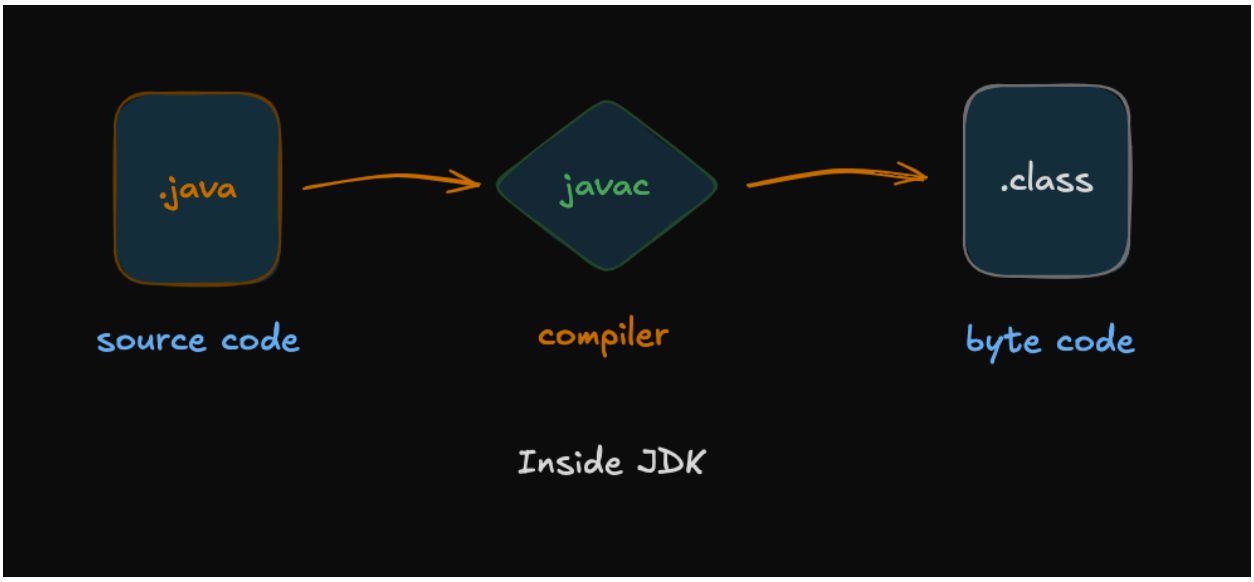
Step 1 - Compilation :

- Each individual `.java` source file is picked up by the **JDK** (Java Development Kit).
- This file contains **human-readable Java code** written by the developer.
- The `javac` **compiler**, a part of the JDK, compiles the source code into **bytecode**.
- This bytecode is stored in a `.class` file.
- **Bytecode is platform-independent** - it is not specific to any hardware or OS.
- The compiled bytecode can now be executed on **any system that has a Java Virtual Machine (JVM)**.

```
# Terminal Command:
"dev_vedant@Vedants-MacBook-Pro Java_Basics %" javac filename.java

# This command compiles the file into byte code.
# This is what our IDE does when we run the program.
```

> 🧠 Why Bytecode?
>
> Bytecode serves as an intermediate representation. It allows Java to be a **"write once, run anywhere"** language - the JVM on each platform takes care of converting bytecode to native machine code.



Step 2 - Execution via JRE :

java launcher:

- The **java launcher** is an **executor** that **invokes the JVM** and passes it the .class file for conversion into machine code.
- We don't need to give `.class` extension as it auto-picks the bytecode file and loads it into a **class loader**.
- **JVM** starts after running this statement.

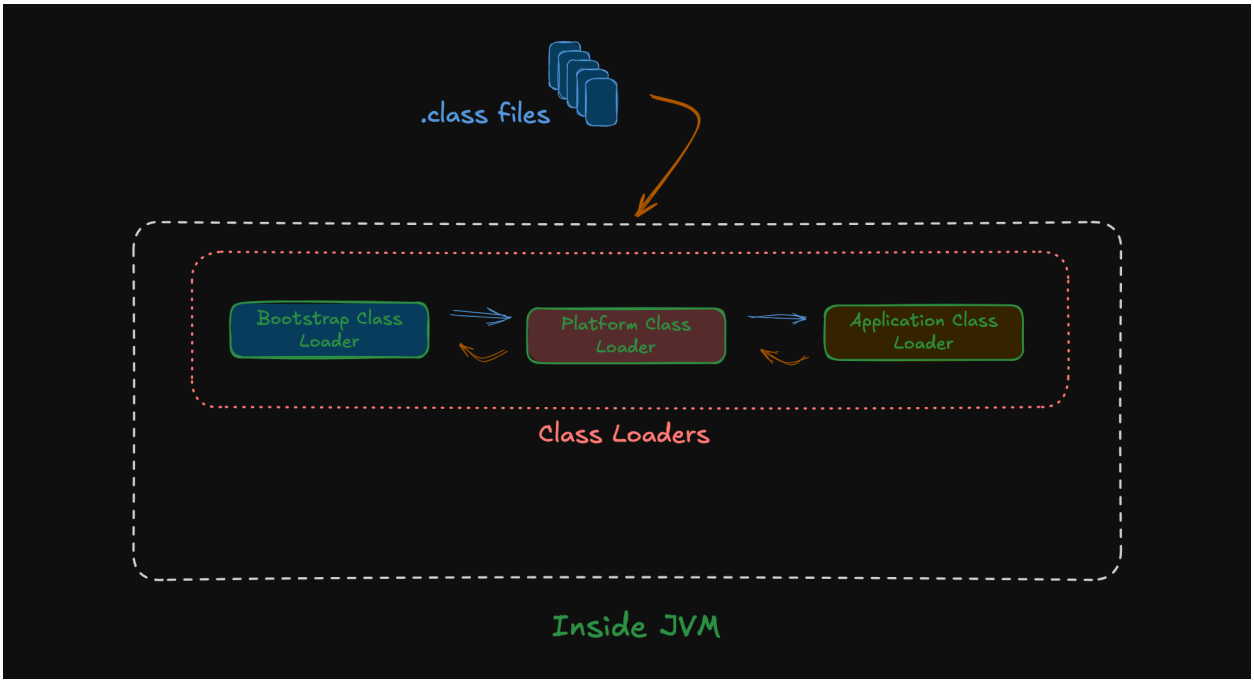
```
# invoking JVM using java launcher:
"dev_vedant@Vedants-MacBook-Pro Java_Basics %" java filename
```

Step 3 - Class Loaders:

- The Class Loader is the part of the JRE that loads Java `.class` files into the memory on demand.

- It forms the first stage of JVM execution process, after the java execution command is invoked.
- The Class Loader **reads the** `.class` **file** (compiled bytecode) from the **file system**, **network**, **or JAR**.
- It **loads the binary data** (byte array) of that class into **JVM memory**, specifically into the **Method Area.**

> The Class Loader creates a **Class object** in the **Method Area** of the memory.
>
> - This `Class` object stores:
> - The constant pool
> - Field and method metadata
> - Modifier info
> - Annotations, etc.
>
> 🔷 This happens **before linking**, and is called **"loading"** - even though it does write some initial data into memory.



JVM has 3 main Class Loaders, each for specific tasks instead of one to make **JVM more efficient** and **performant**.

Bootstrap Class Loader:

- The **Bootstrap Class Loader** is the **parent of all class loaders**. It primordial and has the **first priority** amongst other loaders.
- Its written in **pre-compiled** native code (C,C++).

The **Bootstrap Loader** is responsible for loading only the **required** `java.base` class files.

- These base classes are from the Java Standard Library and are loaded based on the requirements of the program.

These include classes from:

- `java.lang.*` (like `Object`, `String`, `System`, `Thread`)
- `java.util.*`
- `java.io.*`
- `java.net.*`

Aspect	Details
Name	Bootstrap ClassLoader
Type	Native (non-Java)
Loads From	`<JAVA_HOME>/lib/` or module system
Classes Loaded	`java.lang.*`, `java.util.*`, `java.io.*`, etc.
Parent of	Extension ClassLoader
Represented as	`null` in Java

Platform Class Loader:

The **Platform Class Loader** is introduced in **Java 9** as a **replacement for the Extension Class Loader**.

- It is a **child** of the **Bootstrap ClassLoader.**
- It is a **parent** of the **Application ClassLoader**.
- Platform classes reside in the `$JAVA_HOME/lib/modules` file, which contains all the platform modules in Java 9 and later.

✅ **Roles & Responsibilities:**

- Loads classes from **platform modules**, i.e., modules that are not part of `java.base` but are part of the **standard Java platform** — like:
  - `java.sql`
  - `java.xml`
  - `jdk.crypto.ec`
  - etc.
- Delegates **upward** to **Bootstrap ClassLoader** for `java.base` classes like `java.lang`, `java.util`.
- Delegates **downward** to **Application ClassLoader** for user-defined classes.
- It is implemented by `jdk.internal.loader.ClassLoaders$PlatformClassLoader`.

Aspect	Details
Name	Platform ClassLoader

Aspect	Details
Loads	Java platform classes beyond `java.base` like `java.sql`, `java.xml`, etc.
Parent	Bootstrap ClassLoader
Child	Application ClassLoader
Implemented In	Java (`jdk.internal.loader.ClassLoaders`)
Java 9+ Replacement For	Extension ClassLoader

Application Class Loader:

The **Application ClassLoader**, also known as the **System ClassLoader**, is a child of the **Platform ClassLoader**. It is responsible for loading application classes that are outside the scope of both the **Bootstrap** and **Platform** ClassLoaders.

- The Application ClassLoader is the **default class loader** that loads your **application's classes** including the classes you write as well as any **external dependencies** (such as **third-party libraries or JAR files**) you add to your project.

What does it load?

- Classes located in paths defined by the environment variable `CLASSPATH` or the JVM options `cp` / `classpath`.
- This includes your project's compiled output directories, external libraries (JAR files), and modules that are not part of the Java platform.

In detail, it loads classes from the **classpath**, which consists of:

- `.class` files stored in directories
- JAR files
- Modules specified on the **classpath** or module path

Hierarchy:

- The Application ClassLoader is a **child of the Platform ClassLoader**, which itself is a child of the **Bootstrap ClassLoader**.

- It follows the **parent delegation model**, meaning if it cannot find a class, it delegates the loading request upwards.
- The typical delegation chain is:

```
Application ClassLoader → Platform ClassLoader → Bootstrap ClassLoader
```

ClassLoader Delegation Model

What is Delegation?

The delegation model is a hierarchical approach where class loading requests are **delegated upward** in the class loader hierarchy before being handled locally.

How Delegation Works:

1. When a class loader is asked to load a class, it **does not load it immediately**.
2. Instead, it first **delegates the request to its parent class loader**.
3. If the parent is **able to find and load the class**, that class is returned and used.
4. If the parent **fails to find the class**, then the current class loader attempts to load the class itself.

Example in Practice:

- Suppose the **Application ClassLoader** is asked to load a class.
- It will first ask the **Platform ClassLoader**, which in turn asks the **Bootstrap ClassLoader**.
- If none of the parents can load the class, only then does the Application ClassLoader try to load it from the **classpath**.

Why Use Delegation?

- ✅ **Security:** Each class loader is responsible for loading classes from trusted locations. Delegating ensures core Java classes are always loaded by trusted loaders (like Bootstrap), reducing risks.
- ✅ **Avoids Redundancy:** Prevents multiple loaders from loading the same class independently, which could cause conflicts and class-casting issues.

- ✅ **Separation of Concerns:** Different loaders are designed to handle specific types of classes (e.g., core Java classes, platform APIs, application code), improving **modularity** and **performance**.
- ✅ **Predictability:** Ensures consistency-if a class like `java.lang.String` is loaded, it's always loaded by the Bootstrap loader, no matter who asked for it.

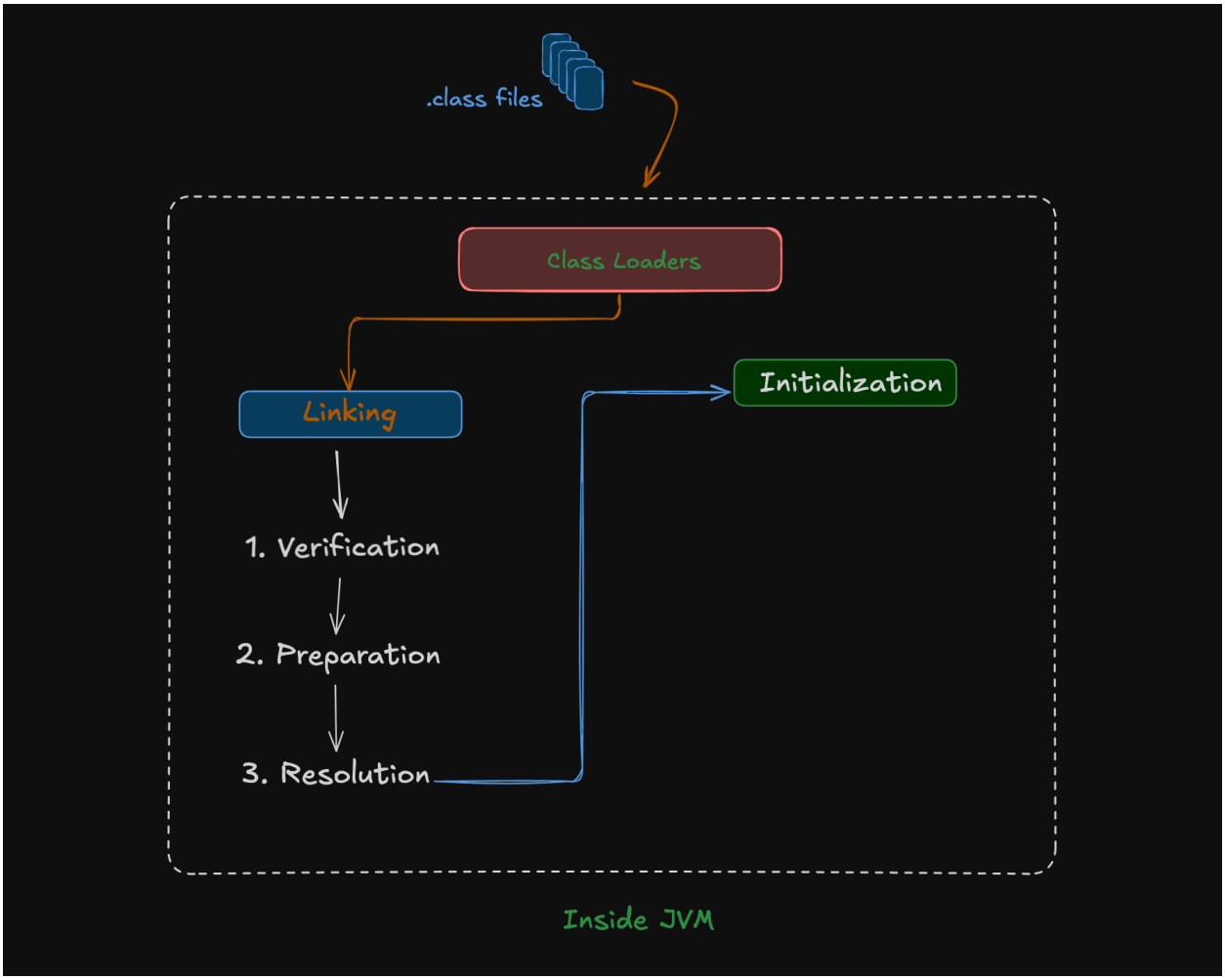
Delegation Chain Overview:

```
Application ClassLoader
      ↓
Platform ClassLoader
      ↓
Bootstrap ClassLoader
```

Step 3 - Linking and Initialization:

These **two processes** are crucial for **preparing bytecode** before it is executed by the JVM.

During this stage, the **bytecode** is analyzed to **build a structural blueprint** (or skeleton) of the **classes**, including their **fields**, **methods**, and **execution order**. This ensures that the JVM knows what **needs to be executed**, where to **locate class references**, and how to resolve dependencies when the actual execution begins.



Linking:

Linking is the second step in class loading where it prepares the class files (byte code) by verifying, organizing, assigning memory and resolving references.

There are 3 sub-steps in Linking:

Verification:

- Ensures the bytecode follows JVM specifications.
- Checks for malicious or inconsistent byte code.
- Checks Type safety.
- **Purpose:** So that faulty or malicious code doesn't crash or hijack the JVM.

Preparation:

- **Allocates memory** for class level variables (i.e., **static variables**).
- **Reserves space** in **Method Area** for these static variables.
- **Initializes** these variables with **default values** (`0` , `false` , `0.0,` etc. for primitive types, `null` for reference types).

Resolution:

- Is Final step of linking where **symbolic references** are resolved and classes are **loaded** from the **class locations**.
- Bytecode doesn't store the actual classes, it just contains their locations like for;

```
String s = new String();
and MyClass myclass = new MyClass();
```

🔷 the byte code looks something like this:

```
java/lang/String - location of String class.
com/Vedant/MyClass - location of written class.
```

- The linker looks up to the **Class loader** to find which class file contains this class and loads it into the **method area** for the JVM to use **during interpretation**.

Initialization:

- The **final step** in the class lifecycle after linking.
- Responsible for **assigning actual values** to **static variables** and executing static blocks.
- Ensures the class is fully prepared **before any** `main()` **method or object constructors** run.

- **Static variables and static blocks are executed exactly once per class**, in the order they appear in the source code.
- If there are no static variables or blocks, the JVM still marks the class as initialized to allow further processing.
- Initialization is **triggered by**:
  - Accessing a static field or method.
  - Creating an object using the `new` keyword.
  - Using reflection to inspect or invoke class members.

Example:

```
class Test {
    static int x = initializeX();  // 1st: initializeX() method runs first
    static int y = 20;             // 2nd: y is assigned 20
    static {
        System.out.println("Inside static block");  // 3rd: static block executes last
    }

    static int initializeX() {
        System.out.println("Initializing x");
        return 10;
    }
}
```

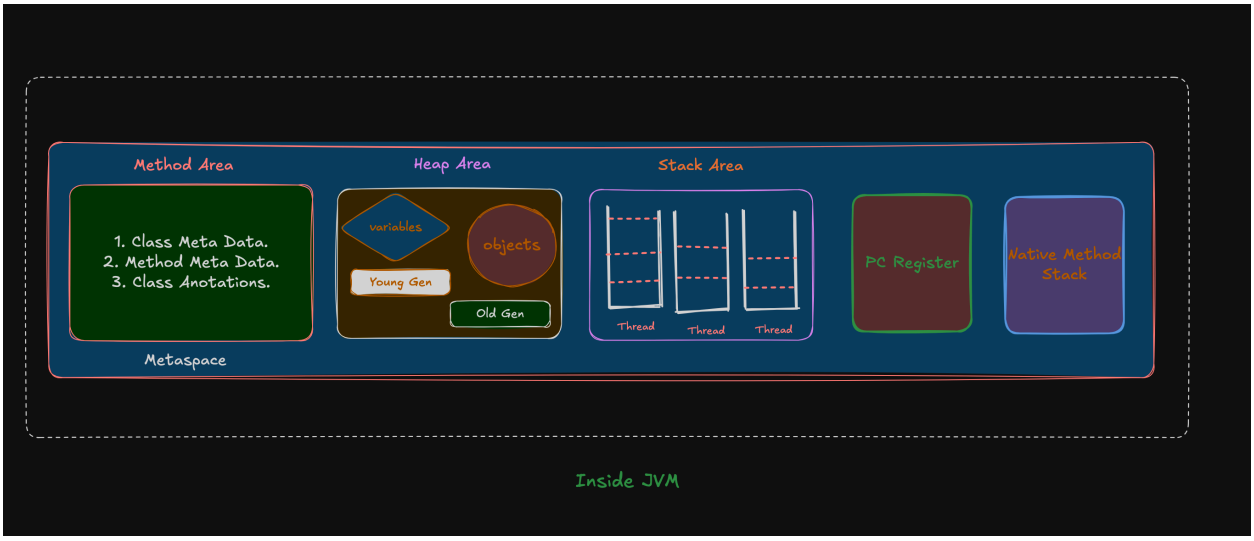
Output during initialization:

```
Initializing x
Inside static block
```

- Now the Classes are ready to be processed and they are now loaded into the Memory Area.

Step 4 - Memory Area :

During runtime, the **JVM borrows memory** from the OS and allocates it to different memory areas. Each area is responsible for storing a specific kind of data — from class metadata to method stacks and native language stacks.



Method Area (MetaSpace):

> **Method Area:** is a conceptual specification (mental model) in the JVM that defines what kind of data should be stored — such as class structures, method definitions, runtime constant pools, and static variables.
>
> **MetaSpace:** is the actual **code implementation** of the Method Area introduced in Java 8. Unlike the earlier PermGen space(not scalable), MetaSpace uses **native memory** (outside the heap), is **scalable**, and stores **class metadata** and **static variables**.

The MetaSpace is accessible by all threads.

Stores:

- Class Meta data.

- Method Meta data.
- Static Variables.
- Runtime Constant Pool.

1. **Class Meta Information :**

- Fully qualified class names.
- Superclass names,
- Implemented Interface.

```
// Example :
public class Car extends Vehicle implements Serializable(){
    int wheels = 4;   - // instance variable
    static int CarsCreated = 0;   - // static variable
}

// JVM stores:
/*
* Car → class name
* Vehicle → superclass
* Serializable → Interface.
*/
```

2. **Method Meta Information :**

- Method Name.
- Return Type `void`, etc.
- Parameter Types.
- Access Modifiers.

```
// Example:
public void startEngine(String keyType) {....}
```

```
public static int getCarCount() { return carsCreated; }

// JVM stores:
/*
* Method name → startEngine and getCarCount.
* Return Type → void and int respectively.
* Parameter type → String for keyType.
* Access Modifiers → public.
*/
```

3. **Static Variables:**

- **Static variables** are **shared across all instances** of a class. To **avoid duplication**, they are stored in the **Method Area** (or MetaSpace in Java 8+) rather than within individual object memory.

```
// Example:
public class Test(){
    static int count = 0;
}

// count = 0 set in method area accessible to all instances.
```

4. **Runtime Constant Pool (RCP):**

Each class has its own runtime constant pool in method area. It stores:

- String literals.
- Final constants.
- Reference to symbolic references.

```
final int x = 20;
String name = "Vedant";
```

```
// Vedant set in constant pool.
// 20 inlined as constant value.
```

Heap Area:

Heap Area is the **runtime data area** from which memory for all **objects** and **arrays** is allocated. This is **shared** by **all threads**.

Heap Structure is divided into two parts:

Young Generation (Young Gen) :

- This is where **new instances** are **first** created.
- This is further divided into two spaces:
  - **Eden Space:** freshly created objects.
  - **Survivor space (S0/S1):** instances that have **survived** few **Minor GC** cycles.

Old Generation (Old Gen) :

- Here **Long-lived objects** are stores (survive multiple GC sweeps).
- These objects are **more stable** that **Young Generations** as they persist for longer times.
- Requires **Major GC cycles** that are expensive in terms of time and resources, but **not that frequent**.

Object Lifecycle of Objects in Heap :

Young Generation (Volatile / Unstable) :

- Objects are **freshly created**.
- Most Objects are **volatile, unstable** and die quickly because they are **short lived** (temp variables, etc).
- **Frequent Minor GC cycles** for **Eden** and **Survivors**. Not that resource intensive to run Minor GC's.
- If an object survives several Minor GC cycles, its **promoted to Old Gen**.

Old Generation (Stable) :

- Only Long-Lived objects make it here, those which **survive Young Gen**.
- They are **assumed** to be **permanent** and to last throughout the **Application's Life.**
- **Infrequent Major GC cleans** happen to remove **stale Instances** but not that frequently.

JVM Stack Area :

The **JVM Stack** is a **thread-specific** memory area — meaning **each thread** in the JVM gets **its own stack**. It stores **frames**, which are created **each time a method is invoked**.

- A **frame** contains:
  - Local variables (ints, references, etc.)
  - Operand stack (for intermediate calculations)
  - Method call metadata (like return address)

When the method finishes execution, its frame is **popped off the stack**.

The Frame is pooped off when the method completes execution.

- Work on **LIFO** structure. Meaning the **latest frame** is **resolved** first then popped off.
- **Non Heap:** only stores references and not the complete arrays or objects.

Program Counter Register (PC):

The PC Register is a small memory area that is allocated to each thread, which **stores** the **address** of the **next execution instruction** into the **currently executing instruction** of a method in the bytecode.

- Each thread has its own PC register.
- The PC Register holds the address of the **next executing instruction** (next line).

- For **non-native** methods (written in Java), the PC register **points** to the **JVM bytecode instructions** currently being executed.
- For **Native methods** (written in C,C++), the PC register value is **undefined** cause the Native Method Stack is responsible for executing Native Code.

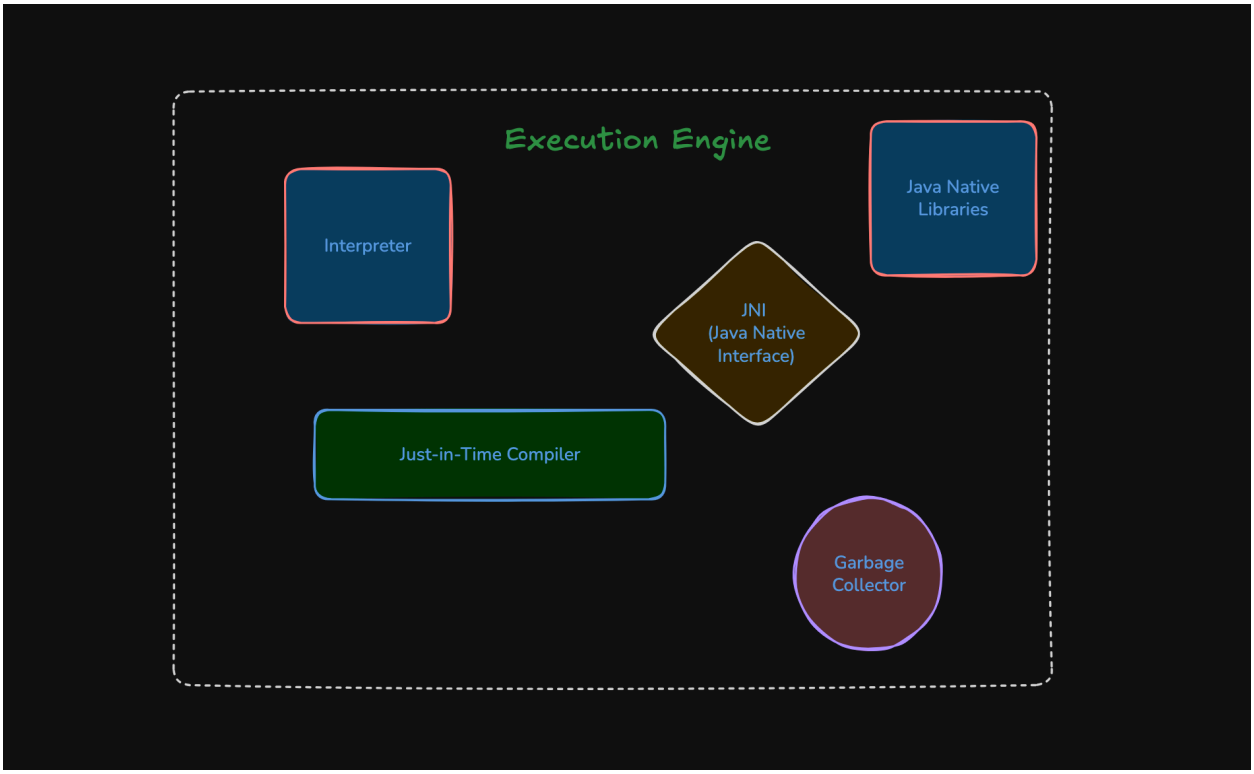
Native Method Stack:

The **Native Method Stack** is a special memory area used to support **native methods** — that is, methods written in **languages other than Java** (like C or C++), which Java can call through the **Java Native Interface (JNI)**.

- Each Thread has a Native Method Stack.

Step 5 - Execution Engine

The Execution Engine is the component of the JVM responsible for **executing the bytecode instructions** of a Java program.



- **How it works:**

- It **reads** the **bytecode loaded** into the **Method Area** and interprets or compiles it into **machine code** that the CPU can run.

- **Two main components:**
  1. **Interpreter:**
     - Reads and executes bytecode instructions **one at a time (line by line)**.
     - **Slower** but allows quick startup and easy debugging.
  2. **Just-In-Time (JIT) Compiler:**
     - Monitors bytecode execution to identify **"hot spots"** (frequently executed code).
     - **Compiles** these hot spots into native machine code for **faster execution**.
     - Improves performance over time by avoiding repeated interpretation.
- **Other features:**
  - **Adaptive optimization:** JVM dynamically decides when and what to compile.
  - **Garbage Collection:** While technically separate, the Execution Engine coordinates with the GC to manage memory.
- **Execution context:**
  - Uses memory areas like the **PC Register**, **JVM Stack**, **Heap**, and **Method Area** during execution.
  - Invokes native methods using the **Native Method Stack**.